Contents

Aegis-SOC — Complete Project Dossier **1**
1. Identity . . . 1
2. The problem it solves . . . 1
3. Architecture . . . 2
4. Detection coverage (13 rules) . . . 2
5. Measured results . . . 3
6. The AI threat model (the distinguishing contribution) . . . 3
7. Security assurance . . . 4
8. CI/CD . . . 4
9. API surface . . . 4
10. Limitations (stated, not discovered) . . . 5
11. Future work . . . 5
12. Repository map . . . 5
13. Reproduce everything . . . 5

Aegis-SOC — Complete Project Dossier

One-page reference for everything: what it is, how it works, what it measured, what it costs, and where each claim is evidenced. Deep detail lives in `REPORT.md` (architecture + evaluation) and `SECURITY-CICD-REPORT.md` (pipeline + security + resources).

---

1. Identity

Field	Value
Project	Aegis-SOC — cloud-native, AI-assisted security operations platform
Course	Advanced Cloud Computing (ECC 6103 / EAI 6304 / EAS 6304 / ESD 6304 / MCA20-E304C / ECS 6302)
Author	Rudra Pratap — M.Tech, Semester 3
Repository	`github.com/Rhack-INDIA/aegis-soc`
Live demo	`http://3.106.151.93` (AWS EC2 `t3.micro`, ~$7.60/month)
Container image	`ghcr.io/rhack-india/aegis-soc:latest` (public)
Stack	Python 3.12 · FastAPI · Pydantic · Docker · SQLite/Firestore · GitHub Actions
Recurring cost	**$0** on the Cloud Run reference design · **~$7.60/month** on the EC2 demo instance

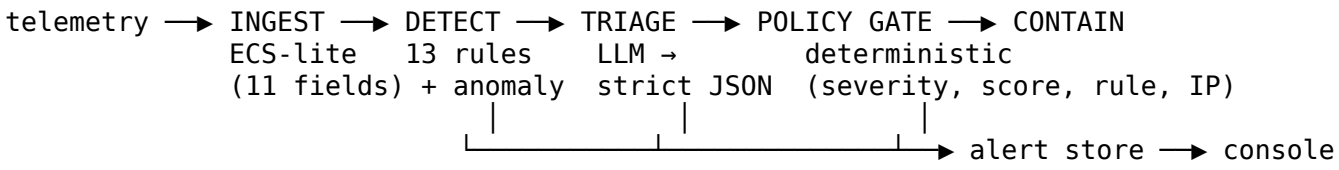
---

2. The problem it solves

A SOC runs a four-step loop: collect telemetry, detect behaviour, triage what fired, respond. Alert volume grows faster than analyst headcount, and commercial SIEM pricing scales with gigabytes ingested rather than with risk. The project asks: **what is the largest credible SOC that fits in a $0 cloud budget, and what does that constraint force?**

---

3. Architecture



Layer	Implementation	Decision it encodes
Ingest	POST `/ingest` → 11-field ECS-lite subset, allow-list filter	Ground-truth labels cannot leak into detection
Detect (rules)	13 rules mapped to ATT&CK, 2 stateful (failure windows)	Precision + explainability
Detect (anomaly)	4-feature statistical baseline trained only on benign traffic	Catches behaviour with no signature
Fusion	One alert per distinct technique	The bug that lost a technique when only the top-severity match was emitted
Triage	LLM → Pydantic-validated JSON (verdict, severity, technique, confidence, narrative, action)	Prose, not authority
Policy gate	severity ≥ HIGH · score ≥ 0.80 · rule match · source IP present	The model cannot authorise containment
Store	SQLite locally, Firestore on Cloud Run (pluggable)	Cloud Run is ephemeral; alerts must survive
Console	Single-page, no build step, served by the same container	Demonstrable in 90 seconds

---

4. Detection coverage (13 rules)

Rule	Detection	ATT&CK	Severity
R001	SSH brute force from one source (≥5 failures / 120 s)	T1110	high
R002	Successful login after ≥5 failures	T1078	critical
R003	Local account creation	T1136.001	high
R004	`authorized_keys` modification	T1098.004	critical
R005	Cron persistence	T1053.003	medium
R006	Reverse shell patterns	T1059.004	critical
R007	base64 payload piped to shell	T1140	high
R008	Log tampering	T1070.002	high
R009	Sudoers modification	T1548.003	critical
R010	`/etc/shadow` access	T1003.008	high
R011	Web traversal / injection	T1190	medium
R012	Download-and-execute	T1105	high
R013	Archive staging	T1560	medium

RAG corpus holds 14 ATT&CK techniques with descriptions and mitigations; retrieval is IDF-weighted term overlap (no embeddings — PyTorch would exceed the 0.5 GB image budget).

---

5. Measured results

Corpus: 624 events — 600 benign, 18 attack, 6 evasion — across 13 techniques. Regenerate with `python scripts/eval.py`.

Metric	Value	Meaning
Technique recall	**13/13 (1.000)**	Pipeline correctness — circular, see next row
Precision / F1	1.000 / 1.000	15 TP alerts, 0 false positives on benign replay
Evasion attribution	**1/6 (0.167)**	The honest ceiling of signature detection
Prompt-injection ASR	**58.3% → 0.0%**	Undefended → defences D1–D5 (12 payloads)
Throughput	**411.7 rps**, 0.0% errors	20 concurrent clients on `/ingest`
Latency	p50 23.9 ms · p95 154.2 ms · p99 249.8 ms	Same run, single worker
Memory	53.2 MiB idle → 54.3 MiB peak	512 MiB request is ~10× oversized
Tests	**30**	14 behavioural + 16 API-contract

The 1.000 is not a quality claim: corpus and rules were written together. The evasion set exists precisely to give a non-circular number.

---

6. The AI threat model (the distinguishing contribution)

Every field in a log line is attacker-written text, so an LLM reading logs consumes untrusted input that can carry instructions. Twelve payloads were hidden in attacker-controlled fields (`cmdline`, `User-Agent`), each attempting to make a HIGH-severity detection be reported as benign.

Defence	Mechanism
D1 Spotlighting	Untrusted content delimited; system prompt states log text is data, never instructions
D2 Sanitisation	Control chars stripped, delimiters escaped, fields truncated
D3 Schema validation	Response parsed into Pydantic; non-conforming output rejected, not repaired
D4 Action separation	The model cannot contain; a deterministic policy gate decides
D5 Critic override	A `false_positive` verdict against a high/critical rule is escalated

Result: 7/12 hijacked undefended, 0/12 defended. D4 is load-bearing — a successful hijack still cannot trigger a block.

---

7. Security assurance

Finding	Severity	Status
3 × `starlette 0.41.3` CVEs (Range DoS, StaticFiles SSRF/NTLM, form-limit DoS)	HIGH	Fixed (fastapi 0.146 / starlette 1.9)
44 × `util-linux` OS packages	HIGH	Accepted — parsed Trivy JSON: **every one has no fixed version**
`pytest` PYSEC-2026-1845	MEDIUM	Fixed (≥9.0.3, moved to dev requirements)
Dockerfile DL3025 shell-form CMD	MEDIUM	Fixed (exec-form entrypoint)
Dockerfile DL3066 non-numeric USER	LOW	Fixed (`USER 10001`)
4 × bandit B110/B112 defensive `try/except`	LOW	Accepted, CI fails on MEDIUM+
86 lint findings (24 under project policy)	INFO	Fixed — 0 remaining

Totals: **7 fixed, 4 accepted with stated residual risk, 0 open.** Secret scan: gitleaks over full history, no leaks (and it initially reported "no leaks" having scanned zero commits — documented as a methodological finding).

---

8. CI/CD

```
git push → ci (6 gates) → release (build + smoke test + publish GHCR)
                                  ↓
                    host pulls :latest (Watchtower)
```

Gate	Tool	Policy
Lint	ruff	`E,F,W,I,UP,B`
Tests	pytest	30 tests
SAST	bandit	fail on MEDIUM+
Dependencies	pip-audit	runtime **and** dev sets
Secrets	gitleaks	full history, `.gitleaks.toml` allowlist
Container	hadolint + Trivy + smoke	Trivy fails only on **fixable** HIGH/CRITICAL

Deploy is `workflow_dispatch` only, re-runs all gates, and uses Workload Identity Federation (keyless) for GCP. Dependabot covers pip, docker and actions.

---

9. API surface

Method	Path	Purpose
POST	`/ingest`	One event, array, or `{"events":[…]}`

Method	Path	Purpose
GET	`/api/alerts?limit=`	Alert queue, newest first
GET	`/api/alerts/{id}`	Full alert incl. triage
POST	`/api/alerts/{id}/triage?defenses=`	Re-triage with defences on/off
POST	`/api/alerts/{id}/block`	Deterministic policy gate
GET	`/api/stats` · `/api/health` · `/api/version`	Counters, mode, deployed SHA
POST	`/api/simulate/attack` · `/api/simulate/injection` · `/api/reset`	Demo helpers

---

10. Limitations (stated, not discovered)

1. Synthetic corpus, single host; no Windows telemetry, no network sensors, no real adversary.
2. Triage runs a deterministic mock unless `GEMINI_API_KEY` is set — the UI labels it `simulated`.
3. Containment is simulated (`APPLY_BLOCKS=simulated`); no edge rule is created.
4. The demonstrated instance is AWS EC2 (~$7.60/month); the **$0 design is Cloud Run**, blocked by a closed GCP billing account.
5. Brute-force counters are per-instance, so thresholds scale with instance count.
6. No authentication on the console; HTTP only. Demo-only.
7. Cold-start latency was never measured (`docker restart` blocked by an approval prompt).

---

11. Future work

Live-model measurement; pySigma-compiled SigmaHQ rules; detections for the evasions that succeeded (`openssl`, systemd timers, in-place truncation); detection state moved to Firestore; Pub/Sub between ingest and triage; an uptime SLO.

---

12. Repository map

```
app/          schema · store · detect · triage · injections · main · static/index.html
lab/            gen_corpus.py — benign + attack + evasion corpus with ground truth
scripts/      eval.py · loadtest.py · security_checks.sh · ui_smoke_cdp.py · polish_slides.py
tests/          test_aegis.py (14) · test_api_contracts.py (16)
.github/        workflows/{ci,release,deploy}.yml · dependabot.yml
docs/        REPORT · SECURITY-CICD · PRESENTATION · DEMO_SCRIPT · VIVA_QA · DEPLOY_NOTES
                Aegis-SOC-Slides.pptx · PDFs
evals/          results.json · security/* · resource/*
```

13. Reproduce everything

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements-dev.txt
python lab/gen_corpus.py      # corpus + baseline
python scripts/eval.py        # every number in §5
uvicorn app.main:app --port 8080   # console
python -m pytest              # 30 tests
./scripts/security_checks.sh  # the CI gates, locally
```